任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关

前端页面的用户输入，本质是给程序传递「查询/操作条件」，程序拿到输入后，必须去数据库存取对应数据，这就是两者绑定的根本原因。

2. 一个最小业务链路是什么

用户输入参数 → 后端接收 → 构造 SQL → 数据库执行查询 → 返回数据给前端

3. 用户输入是如何进入查询条件的

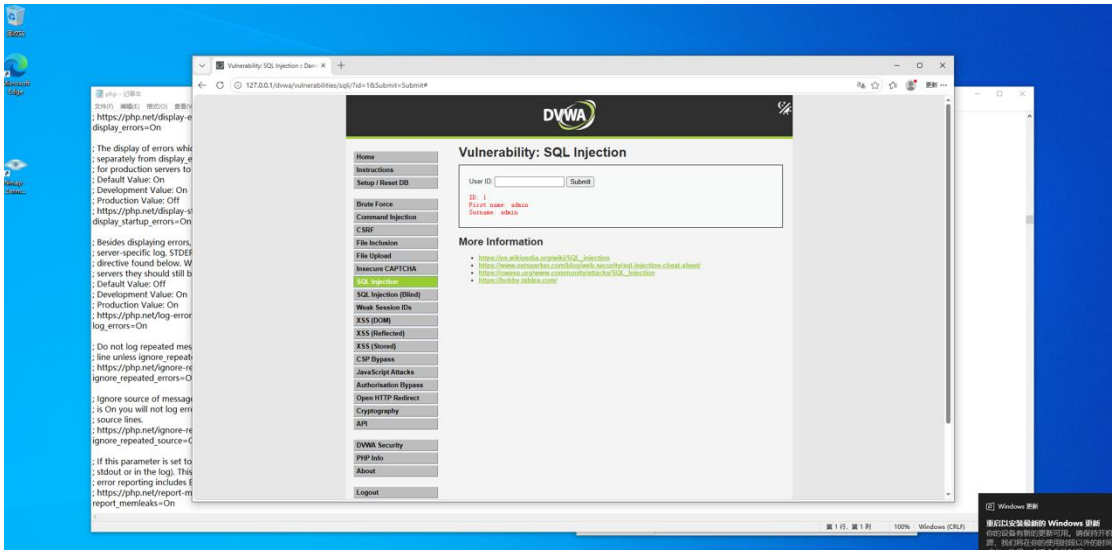
（1. 前端收集用户输入，通过请求传给后端；2. 后端接收输入，存入程序变量；3. 后端将该变量拼接/替换到 SQL 语句的 WHERE 后查询条件位置；4. 原本固定的查询条件，变成携带用户输入的动态条件；5. 组装完成整条 SQL，下发给数据库执行。

4. 为什么这为 SQL 注入创造了条件

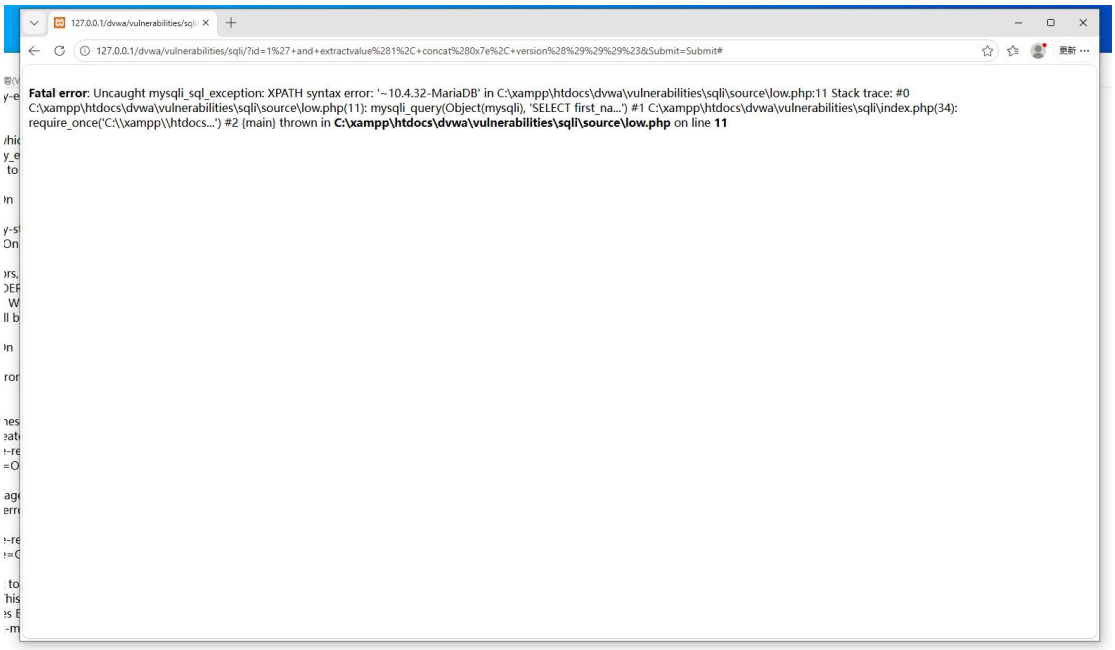
因为后端将用户可控输入拼接进 SQL 查询条件，使输入既可以作为查询数据，又能注入 SQL 语法篡改原有查询逻辑，因此产生了 SQL 注入漏洞条件。

任务 2：DVWA SQL Injection Low 现象记录

普通输入：



异常输入：



页面没有正常显示用户数据，而是直接弹出一段 MySQL 数据库错误信息正常情况无论我输入 1、2 还是 3，页面都应该乖乖返回一个用户的 first name 和 surname。

但现在页面不显示数据，反而报错，而且报错里还包含了数据库的版本信息。

正常查询变化只是换一个 ID，页面内容跟着换一条记录，不会无缘无故变成错误页面。

而这里页面直接变成了数据库调试信息回显，说明输入已经不是简单地“改条件”，而是改变了 SQL 的执行流程，把查询变成了一个会主动抛异常的恶意表达式。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

   最典型的路径是：

   输入 → 拼接 → 解析 → 执行 → 回显/行为差异

举个例子：

后端代码里写了一条 SQL 模板 "SELECT * FROM users WHERE id = '" + 用户输入 + "'"。

用户在页面上输入 1，这个字符串就被直接拼到模板里，变成 ... id = '1'。

然后数据库解析这条 SQL，看到单引号里面的内容当作字符串值，再执行，最后把结果回显到页面上。

这就是输入进入查询结构的全过程——通过字符串拼接，没有任何隔离。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

单个字符本身没有任何攻击能力。

比如 ' 在数据库里就是个普通符号，你存一个名字叫 O'Neil 是合法的。

真正出问题的是：这个 ' 出现在一个特定位置，它提前闭合了原本的字符串边界，导致后面跟着的 OR 等关键词变成了 SQL 指令的一部分。

3. Low 与 Medium 的差异在哪里

DVWA 的 Low 级别：直接拼接，不做任何处理。

Medium 级别：用了 mysql_real_escape_string() 函数，把单引号、双引号等转义掉。

差异看起来是“Medium 更安全”，Low 对任何输入不加区

分地拼接

Medium 试图用转义堵住引号这条路，但没改查询的数据类型，于是数字型注入依然畅通

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

普通注入：页面可能会直接显示数据库中的数据，或者抛出详细的数据库报错信息

5. 为什么盲注和普通注入根因相同，但证据形式不同

因为应用程序对错误的处理方式不同：

有些程序直接把查询结果或报错信息打印到页面上（普通注入如鱼得水）

有些程序把所有错误吞掉，只返回一个固定页面（比如“查询无结果”），攻击者只能通过布尔条件或时间延迟来一点点试探

任务 4：SQL 注入防护清单与解释

输入校验的目的是在数据进入数据库之前，就把明显异常的请求拦截掉。优先采用白名单策略：如果某个参数预期是数字，就直接拒绝任何包含字母、引号、SQL 关键字的输入。这能有效缩小攻击面，让攻击者的恶意 payload 根本到达不了查询逻辑。

输入校验控制的是应用层（前端+后端），属于暴露面缩减。但它不能替代参数化查询，因为校验总有覆盖不到的地方

（比如搜索框本身就允许各种字符），一旦规则写漏了，注入仍然可能发生。而且，黑名单式的过滤（比如去掉 ’、OR 等关键词）几乎总是能被绕过。

三、错误处理（暴露面缩减）

很多注入之所以能成功，是因为应用程序把数据库的报错信息直接扔给了浏览器。攻击者利用这些信息可以推断表名、字段名、SQL 语法结构，甚至直接获取数据。正确的错误处理是把所有原生数据库错误都“吞掉”，只返回一个通用的提示，比如“系统繁忙，请稍后重试”。

错误处理控制的是应用层（后端配置）和输出层，同样属于暴露面缩减——它隐藏了攻击者需要的线索。但它不能替代参数化查询，因为注入本身仍然可以发生（比如盲注），只是攻击者无法从报错中得到直接帮助。

四、最小权限原则（后果范围缩减）

即使前面的措施都失效了，至少还可以限制攻击者能干什么。应用程序连接数据库时，应该使用一个权限极低的专用账号：只对必要的表做必要的操作（通常只有 SELECT、INSERT、UPDATE 中最低限度的那一个），绝不能是 root 或具有 DROP、FILE 等危险权限的账号。

最小权限控制的是数据库层/权限管理层，属于后果范围缩

减——它不阻止注入发生，但能大幅降低注入成功后的破坏上限。比如如果账号是只读的，攻击者最多只能查数据，不能删表、写文件或提权。然而，这并不能替代参数化查询：如果账号本来就能读取敏感表（比如用户密码表），注入仍然会导致数据泄露。

五、额外措施：Web 应用防火墙（暴露面缩减，辅助）

WAF 部署在网络入口层，通过规则匹配拦截常见的注入 payload（如 ’ or 1=1、union select 等）。它可以作为临时或补充防线，尤其适合来不及改造老旧代码的环境。WAF 也属于暴露面缩减，但它不能替代参数化查询，因为规则可以被编码、分段、数据库特性等方式绕过，且对未知变种无效。

总结

把这五条放在一起看，真正能控制根因的只有参数化查询；输入校验和错误处理是在攻击路径上缩减暴露面；最小权限控制的是后果范围；WAF 是额外的暴露面过滤层。正确的做法是：优先保证参数化查询全部到位，然后用其他措施层层加固，而不是反过来。先做 WAF 和错误处理而不改代码，等于把防盗门装在纸墙上。